

Adaptive Campaign Optimization Engine

Using Contextual Bandits

DS 223 — Marketing Analytics | Group Project

Spring 2026 | American University of Armenia

Milestone 1 — Problem Definition

1. Problem Area

Campaign Personalization and Customer Conversion Optimization.

Marketing teams in e-commerce businesses rely on static, rule-based campaign strategies: segment customers by RFM scores, assign a blanket promotion to each segment, and hope for the best. This approach treats all customers within a segment identically, ignores how individuals actually respond to different offers, and never learns from past outcomes. Budget is wasted on promotions that do not convert, while potentially responsive customers receive no intervention at all.

2. Preliminary Research

The inefficiency of static campaign allocation is well-documented in both industry and academic literature. The core limitations are:

- **Blanket discounting erodes margins.** Offering the same discount to an entire segment means subsidizing customers who would have converted anyway, while still missing those who need a different type of incentive entirely.
- **A/B testing does not scale to many actions.** Traditional A/B tests compare two variants with uniform random assignment. When the action space grows (multiple offer types across diverse customer profiles), uniform randomization wastes budget on clearly suboptimal assignments and converges slowly.
- **Predictive models predict, but do not prescribe.** Churn or propensity models identify *who* is at risk or likely to convert, but do not answer *which action* to take for each individual. The gap between prediction and decision remains unaddressed.
- **Contextual bandits close this gap.** Li et al. (2010) introduced LinUCB for personalized news recommendation at Yahoo, demonstrating that contextual bandits outperform context-free approaches by conditioning action selection on user features. The same framework applies directly to marketing: customer features are the context, promotional offers are the arms, and profit is the reward.

This project applies the explore-exploit framework covered in the course (Weeks 4–6: Bayesian A/B testing, epsilon-greedy, UCB1, Thompson Sampling) to a downstream marketing campaign optimization problem. A key advantage of bandit-based optimization is that it is computationally lightweight — requiring only linear algebra operations — making it feasible even for small teams without large infrastructure budgets.

References

1. Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010). *A Contextual-Bandit Approach to Personalized News Article Recommendation*. In Proceedings of the 19th International Conference on World Wide Web (WWW), pp. 661–670. ACM. <https://arxiv.org/abs/1003.0146>
2. Chu, W., Li, L., Reyzin, L., & Schapire, R. E. (2011). *Contextual Bandits with Linear Payoff Functions*. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), pp. 208–214. <http://proceedings.mlr.press/v15/chu11a.html>
3. Auer, P. (2002). *Using Confidence Bounds for Exploitation-Exploration Trade-offs*. Journal of Machine Learning Research, 3, 397–422.
4. Chen, D., Sain, S. L., & Guo, K. (2012). *Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining*. Journal of Database Marketing & Customer Strategy Management, 19(3), 197–208.

3. Specific Problem Statement

Given a stream of customers with observed behavioral and transactional features, select the single best promotional action (from a discrete set) for each customer at each decision point, in order to maximize cumulative profit over time, while learning which actions work best for which customer profiles.

This is a sequential decision-making problem under uncertainty — not a prediction problem. The system must learn from the rewards it observes (conversions, revenue) and improve its policy over time. The setting is an online retail platform where customers arrive, the system chooses a promotional action, and observes whether the customer converts and the resulting profit.

4. Proposed Solution

4.1 Overview

We propose a Campaign Optimization Engine built around a contextual bandit (LinUCB) that receives customer feature vectors, selects a promotional action, observes the conversion outcome and resulting profit, and updates its action-value estimates accordingly. The system balances trying new actions (exploration) with using known high-performing actions (exploitation), improving its strategy continuously.

4.2 Data Strategy (Hybrid)

Real component (customer features): We use the UCI Online Retail II dataset, which contains ~500K transaction records from a UK-based online retailer (2009–2011). From this raw transaction log, we engineer the following customer-level features:

Feature	How It's Computed
Recency	Days since last purchase
Frequency	Number of distinct orders
Monetary value	Average spend per order
Basket diversity	Number of unique product categories purchased
Average order size	Mean items per transaction
Purchase regularity	Standard deviation of inter-purchase intervals

These form the context vector x_i with dimensionality $d=6$.

Synthetic component (response simulation): We simulate how customers respond to promotional actions. Conversion probability is modeled as a logistic function over observed features with action-specific coefficients, plus controlled noise. This approach is standard in bandit research — it is how the original LinUCB paper was evaluated (Li et al., 2010) — and is explicitly disclosed. No claim of real outcome data is made. The simulation allows us to control ground truth for rigorous evaluation.

4.3 Action Space

Action	Description	Cost	Expected Effect
A0: No action	Control — no promotion sent	\$0	Baseline conversion rate
A1: Discount (10%)	Targets price-sensitive customers	Margin reduction	Higher conversion, lower margin

A2: Free shipping	Removes friction for moderate-basket customers	Shipping cost absorbed	Moderate conversion lift
A3: Product recommendation	Personalized suggestion based on category affinity	Low (compute only)	Moderate lift at minimal cost
A4: Bundle offer	Cross-sell complementary items at slight discount	Small margin reduction	Higher basket value if converted

The action space is intentionally kept at 5 actions. More actions require more exploration before convergence, which means more wasted budget during the learning phase. Five actions provides enough variety to demonstrate meaningful heterogeneous treatment effects while remaining tractable for the model to learn within a reasonable simulation horizon.

4.4 Analytical Technique

Primary model: LinUCB (Linear Upper Confidence Bound). For each action, the model maintains a ridge regression estimate of expected reward given the customer's feature vector. At each decision point, it selects the action with the highest estimated reward plus an exploration bonus. The exploration bonus is large when the model is uncertain about an action's effect for a given customer profile, and shrinks as more data is collected. This naturally balances exploration (trying uncertain actions) and exploitation (using actions known to work). The single tunable parameter, alpha, controls how aggressively the system explores.

Baselines: - *Random policy* — uniform random action selection, representing no intelligence. - *Greedy heuristic* — rule-based assignment (e.g., always discount high-RFM users), representing traditional static segmentation.

We compare cumulative reward, per-step regret, and action diversity across all policies.

4.5 Implementation Plan (Microservice Architecture)

The system is implemented as a Dockerized microservice stack per course requirements:

Service	Technology	Responsibility	Container
Database	PostgreSQL	Store customers, actions, rewards, model state	db

ETL / Simulation	Python	Load real features, simulate conversion responses	etl
Model	Python (NumPy)	LinUCB engine, policy selection, parameter updates	model
Backend / API	FastAPI + SQLAlchemy	Serve decisions, log interactions, expose endpoints	app
Frontend	Streamlit	Dashboard: decisions, rewards, policy comparison	front

5. Expected Outcomes

- LinUCB achieves higher cumulative profit than random and heuristic baselines over a simulated campaign horizon.
- The system learns to differentiate actions across customer profiles (heterogeneous treatment effects), assigning different actions to different customer types.
- Exploration decreases over time as the model gains confidence, visible in the action distribution converging.
- A working MVP demonstrating the full decision loop: customer arrives → system selects action → outcome is observed → model updates → next decision improves.

6. Evaluation Metrics

Metric	Definition	Why It Matters
Cumulative Reward	Sum of (revenue – cost) over all decisions	Primary metric; directly measures profit generated
Per-Step Regret	Oracle policy reward minus chosen action reward	Measures learning speed; should decrease over time
Action Distribution	Proportion of each action selected per round	Shows exploration–exploitation balance and convergence

Conversion Rate by Action	Realized $P(\text{convert} \mid \text{action})$ over time	Validates the model learns true response heterogeneity
---------------------------	---	--

7. Scope Boundaries

In Scope (MVP): 5 actions, LinUCB + 2 baselines, single-step decision horizon, hybrid data (real features from UCI Online Retail II, synthetic responses), full Dockerized microservice stack with PostgreSQL, FastAPI, and Streamlit.

Out of Scope (Future Work): Temporal dynamics (customer fatigue, engagement decay), latent unobservable customer traits, multi-step sequential planning, real-world A/B deployment, deep learning models, and distributed data infrastructure.